EXECUTIVE ARCHITECTURE & IMPLEMENTATION REPORT

NorthRiver Data Warehouse & ETL Pipeline

1. Business problem framing

NorthRiver's rapid transactional growth required replacing fragmented transactional reporting with an enterprise data warehouse designed to deliver unified revenue analytics, track customer and product evolution, and eliminate cross-channel operational blind spots. To support executive decision-making, the warehouse needed to consolidate order, product, customer and channel data into a resilient star schema that avoids common reporting traps, such as silent join drops, revenue inflation from fan-outs, and miscalculated unit-rate aggregates, while maintaining full historical accuracy for changing customer and product attributes.

2. End-to-end architecture overview

Source data comes from the NorthRiver PostgreSQL OLTP database (customers, products, channels, orders and order lines). Extract notebooks load it into staging tables in Databricks, where data is cleansed, typed and normalised. Databricks transform and load notebooks then enforce surrogate key mapping and apply Slowly Changing Dimension (SCD Type 2) logic while building the analytics layer in the **northriver_warehouse** schema. A Databricks job, **northriver_nightly_pipeline**, runs the extract, transform and load tasks in order every night at 02:00 (Europe/Berlin). Finally, Power BI connects to the dimensional model and serves interactive revenue dashboards for executive leadership.

3. Core dimensional design decisions

Design pillar	Implementation strategy and rationale
Fact table grain	Declared strictly at the individual order line item level (**fact_orders**). This atomic grain allows complete flexibility for multi-dimensional rollups across dates, products, channels and customers without loss of detail.
Star schema dimensions	Four dedicated dimensions (**dim_customer**, **dim_product**, **dim_channel**, **dim_date**) centred around fact_orders. Dimension keys use integer surrogate keys (**_sk**, for example customer_sk and product_sk) to isolate warehouse reporting from source-system natural key changes.
SCD Type 2 implementation	Applied to **dim_customer** (for example segment and region) and **dim_product** to track historical changes. Validity is managed with **effective_from**, **effective_to** and an **is_current** flag, binding each order to the customer and product state at the time of purchase. dim_channel and dim_date are stable lookups and are not versioned.
Pipeline loading order	Strict dependency sequence: dimensions load first (dim_channel, dim_product, dim_customer, dim_date) to generate or maintain surrogate keys. fact_orders loads last, looking up the correct surrogate keys to enforce referential integrity. The Databricks job enforces this order.

4. Pipeline validation & integrity verification

Data quality and audit checks passed

- **Grain and reconciliation:** verified that COUNT(*) in fact_orders matches the source line-item counts, while COUNT(DISTINCT order_id) correctly isolates unique sales orders.
- **Join completeness:** audit queries comparing COUNT(*) against the count of rows matched to a dimension key confirmed zero silent row drops across dimensional joins. Unmatched late-arriving orders were preserved through LEFT JOIN handling under NULL dimension values.
- **Fan-out prevention:** validated that surrogate key joins operate on a strict 1:1 or 1:Many relationship, preventing duplicate-key fan-outs and ensuring revenue totals are not artificially inflated.
- **Aggregate integrity:** reconciled SUM(revenue) against the source and confirmed that rates such as unit_price are aggregated with AVG(), never summed.
- **SCD Type 2 behaviour:** changing a customer attribute in the source and rerunning the pipeline closed the old version and inserted a new current version.

- **Automation:** recent runs of northriver_nightly_pipeline completed successfully across the extract, transform and load tasks.

5. Limitations & future roadmap

Current limitation: dimension synchronisation is batch-scheduled, which can cause transient unmatched key gaps when order batches outpace dimension updates.

Next improvement: introduce automated late-arriving dimension handling with fallback surrogate keys (for example -1 for Unknown / In-Flight), expand the staging layer with automated schema-drift detection in Databricks, and implement customer cohort retention metrics within the Power BI layer.